Introdução ao Python

Aula de revisões

Pedro Gasparinho

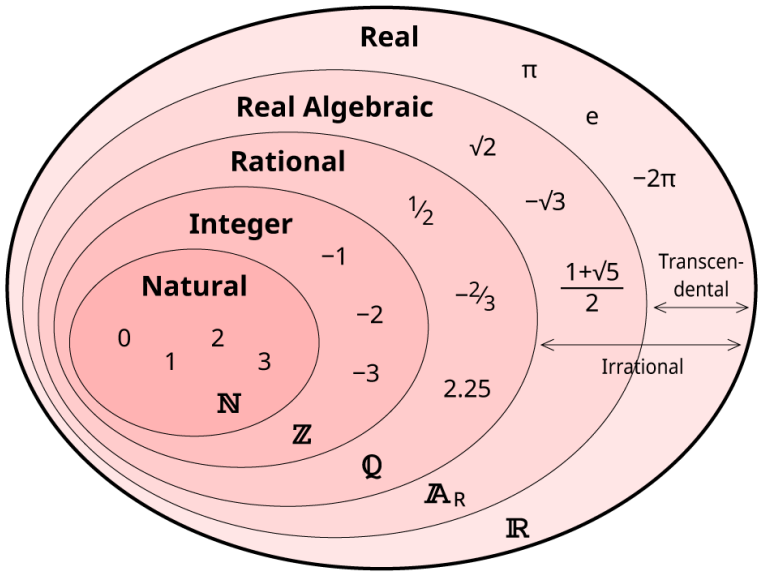
April 15, 2025

Primeiro Programa

```python
print('Hello World!')
```

Python

- *print* - Função
- *'Hello World!'* - Valor do tipo string



- Existe a necessidade de agrupar números com dadas características em conjuntos diferentes.
- Estes conjuntos são normalmente construídos de forma incremental.
- No entanto, quando necessário é possível restringir o conjunto ou subconjunto a usar.

Tipos de dados em Python

- Similarmente à Matemática, em Python e noutras linguagens de Programação, existem conjuntos de valores, denominados tipos de dados.
- No entanto, estes não se limitam apenas a números, existem também formas de representar texto, verdadeiros ou falsos, entre outros.
- Um valor pode ter diferentes representações, mas cada representação apenas está associada a um tipo de dados. Por exemplo, 1 é um valor inteiro, enquanto 1.0 é considerado real.

Tipos de dados em Python

- Números Inteiros (Integers)
  - Denominado *int*
  - Exemplos: *1*, *-4*, *0*
- Números Reais (Floats)
  - Denominado *float*
  - Exemplos: *1.0*, *-6.2*, *3.1415926*
- Valores lógicos (Booleans)
  - Denominado *bool*
  - Exemplos: *True*, *False*

Tipos de dados em Python

- Texto (Strings)
  - Denominado *str*
  - Exemplos: *'1'*, *"Hello, world"*, *""""Olá""""*
- Listas
  - Denominado *list*
  - Exemplos: *[1, 2, 3]*, *[]*, *["Olá", "Olé"]*, *["Olá", 1, ["ok"], true]*
- Entre outros...

Algumas notas sobre Floats

- Não é possível representar fidedignamente todos os números reais, como por exemplo, os números com dízimas infinitas, já que os computadores têm espaço de armazenamento finito.
- Outro exemplo clássico é somar *0.1* com *0.2*, que resulta em *0.30000000000000004*.
- Isto deve-se à norma IEE 754, usada para representar números reais, mas que não será discutida nestes materiais.

Algumas notas sobre Strings

- São obrigatoriamente delimitadas ou por apóstrofes ('...') ou por aspas ("...").
- Escolher as aspas como delimitador permite utilizar os apóstrofes dentro do conteúdo da string e vice-versa.
- Para além disso, é possível usar o mesmo delimitador 3 vezes seguidas para permitir que a string tenha múltiplas linhas. O que geralmente é utilizado para comentários.

Algumas notas sobre Listas

- As listas servem para guardar múltiplos valores de forma ordenada e que permite repetições
- Em Python, ao contrário de outras linguagens, as listas podem conter valores com diferentes tipos de dados.

- Mostrar a diferença entre o ficheiro e a consola

Expressões aritméticas

Um dos usos mais simples das linguagens de Programação é como calculadora, através dos seguintes operadores aritméticos básicos:

- $+$ representa a soma. Ex:
  - $2 + 2$, que é igual a 4
  - $-1.2 + 3.6$, que é igual a 2.4
- $-$ representa a subtração. Ex:
  - $3 - 1$, que é igual a 2
  - $5.3 - 1.8$, que é igual a 3.5
- $*$ representa a multiplicação. Ex:
  - $5 * 2$, que é igual a 10
  - $2.9 * 3$, que é igual a 8.7

Expressões aritméticas

- / representa a divisão real. Ex:
  - 4/2, que é igual a 2.0
  - 5.4/0.25, que é igual a 21.6
- // representa a divisão inteira. Ex:
  - 4//2, que é igual a 2
  - 5//2, que é igual a 2
- % representa o resto da divisão inteira. Ex:
  - 4//2, que é igual a 0
  - 5//2, que é igual a 1
- $**$ representa a potência. Ex:
  - $2 * *4$, que é igual a 16
  - $9 * *0.5$, que é igual a 3.0

Notas sobre expressões aritméticas

- Tal como na Matemática, a divisão por 0 não está definida em Python e caso seja efetuada irá produzir um **erro**.
- A divisão real produz obrigatoriamente um valor do tipo **float**
- Tanto o resto como a divisão inteira produzem valores do tipo **int**
- O tipo de dados do resultado da potência **depende** do tipo de dados da base e do exponente, se pelo menos um deles for float o resultado será float, se ambos forem inteiros será inteiro.

Biblioteca Math

O Python por si só oferece as operações referidas anteriormente, no entanto existe uma biblioteca de Matemática que contém diversas constantes, como o pi ou o número de neper, para além de funções como o logaritmo, a raíz quadrada (alternativa à potência de expoente 1/2), entre outros.

Atribuição a variáveis

Escreva um programa em Python que calcule a distância eucliadiana entre os pontos (3, 4) e (7, 1). A fórmula para dois pontos (x_1, y_1) e (x_2, y_2) é:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Talvez a solução mais intuitiva seja:

Python

```python
import math

print(math.sqrt((3 - 7)**2 + (4 - 1)**2))
```

Repetição de código

E se nos pedirem para calcular a distância entre outros dois pontos? Simples, fazer copy paste!

No entanto, isto introduz alguns problemas:

- A fórmula da distância é relativamente simples, mas copiar pedaços de código mais complexos e com múltiplas linhas torna o código difícil de ler.
- Se detetarmos um erro numa fórmula seria necessário substituir em vários locais, o que não é desejável, e inclusivamente ficar a faltar em algum sítio.

Qual será a melhor solução?

Uma solução mais adequada seria:

Python

```python
import math

def dist(x1, y1, x2, y2):
    return math.sqrt((3 - 7)**2 + (4 - 1)**2)

print(dist(3, 4, 7, 1))
print(dist(0, 0, 0, 1))
```

No entanto, a melhor solução seria:

Python

```python
import math

def dist(x1, y1, x2, y2: float) -> float:
    """ Calcula a distancia eucliadiana entre dois pontos
        x1, y1 - Coordenadas do ponto 1
        x2, y2 - Coordenadas do ponto 2
        Retorna a distancia como numero real """
    return math.sqrt((3 - 7)**2 + (4 - 1)**2)

print(dist(3, 4, 7, 1))
print(dist(0, 0, 0, 1))
```

- Ordem dos parametros importa
- Os nomes não interessam, mas por boa prática deve ser algo alusivo ao comportamento esperado
- Para quê funções? Exemplo do botão na calculadora (i.e. não queremos um botão que calcule especificamente ln(1+5/2))

Exercício

Escreva um função em Python que dado uma nota de 0 a 100 imprime na consola se é positiva ou negativa:

```python
def is_positive_grade(n: int) -> str:
    if round(n) >= 50:
        print("Teve positiva")
    else:
        print("Teve negativa")
```

Python

Em Python é possível expressar alternativas através das keywords *if*, *elif* e *else*. Um bloco condional representa um conjunto de alternativas associadas entre si. Cada bloco começa obrigatoriamente com um único *if*, no entanto é possível omitir tudo o resto (isto representa não fazer nada se a condição original for falsa). Cada bloco condional pode ter 0 ou mais *elif*'s, consoante o número de casos que queiramos distinguir. Já o *else* representa o caso em que todas as outras condições verificadas anteriormente são falsas, ou seja, por bloco condional apenas pode existir no máximo um *else* e deverá ser a última expressão desse bloco condional, pois engloba todos os restantes casos.

TO REMOVE

TODO: Funções

- Pegar num enunciado simples e que peça valores concretos
- Ex: Qual é fibonacci de 4.
- Fazer uma implementação a pensar só no 4
- Depois questionar se quisesse fazer o fibonacci de 8
- Apresentar uma função genérica
- Questionar sobre o que pode estar a faltar
- Introduzir ajudas de tipo e comentários
- Se calhar meter um exemplo super complexo sem comentários e ajudas de tipo
- FALAR SOBRE IDENTAÇÃO E CORPO DAS FUNÇÕES

TODO: While vs For

- Soma de randoms entre 1 a 10 até 100 ou Dice roll = 6 por ex.
- User input

```python
x = "hello" #hello
y = f"world {x}" #hello {world}
## Alice is great
name = "Alice"
greeting = f"Hello {def name.upper() while bool}!"
```

Python

Python

```python
def fibonacci(n):
        return 1

def fibonacci(n: int) -> int:
        return 1

x = lambda a : a + 10
```

Python

```python
import os
import sys

def fibonacci(n):
    """Generate Fibonacci sequence up to n."""
    a, b = 0, 1
    while a < n:
        yield a
        a, b = b, a + b

class SampleClass:
    """A sample class demonstrating Python keywords."""

    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello, {self.name}!"
```

Python

```python
if __name__ == "__main__":
    try:
        num = int(input("Enter a number: "))

        if num < 0:
            raise ValueError("Number must be non-negative")

        for val in fibonacci(num):
            print(val, end=" ")

        obj = SampleClass("Python User")

        with open("sample.txt", "w") as file:
            file.write("This is a sample file.")

        print("File written successfully.")
    except ValueError as e:
        print(f"Error: {e}")
    finally:
        print("Execution completed.")
```